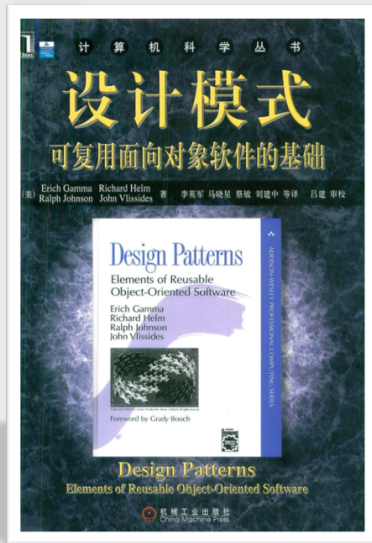




设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

工厂方法（Factory Method）

一、核心定义

定义一个创建对象的接口，但将具体对象的创建延迟到子模块（具体驱动）中完成。

在嵌入式 C 语言中通常表现为：

- 框架规定创建流程
- 具体驱动完成实例构造
- 通过函数指针实现多态

二、结构抽象（通用模型）

框架层（Creator）

```
|  
|-- 调用创建接口  
|  
--> 具体驱动实现创建逻辑  
|  
--> 构造具体设备对象  
--> 绑定操作函数表
```

关键点：

- 框架不直接实例化具体对象
- 创建逻辑由具体实现提供

- 使用与创建分离

三、典型场景（嵌入式）

适合使用场景：

- 驱动框架
- 设备模型
- 总线匹配机制
- 协议栈注册
- 文件系统挂载

不适合：

- 单一固定硬件
- 无扩展需求的小系统

四、案例一：RT-Thread 设备框架

1 抽象设备结构

```
struct rt_device
{
    const struct rt_device_ops *ops;
    void *user_data;
};
```

`ops` = 操作接口（多态）

2 具体驱动实现操作表

```
static const struct rt_device_ops uart_ops =
{
    uart_init,
    uart_open,
    uart_close,
    uart_read,
    uart_write,
};
```

3 驱动初始化函数（工厂方法）

```
int rt_hw_uart_init(void)
{
    static struct rt_device uart_dev;

    uart_dev.ops = &uart_ops;

    rt_device_register(&uart_dev, "uart1", RT_DEVICE_FLAG_RDWR);

    return 0;
}
```

分析

- 驱动负责构建设备对象
- 框架只提供注册入口
- 上层通过统一接口访问

这就是 C 语言环境下的工厂方法。

五、案例二：Linux 驱动 probe 机制

1 抽象接口

```
struct file_operations {
    ssize_t (*read)(...);
    ssize_t (*write)(...);
};
```

2 具体驱动实现

```
static const struct file_operations my_fops = {
    .read = my_read,
    .write = my_write,
};
```

3 probe 函数（工厂方法）

```
static int my_driver_probe(struct platform_device *pdev)
{
    struct my_dev *dev;

    dev = devm_kzalloc(&pdev->dev, sizeof(*dev), GFP_KERNEL);

    dev->fops = &my_fops;

    platform_set_drvdata(pdev, dev);

    return 0;
}
```

分析

- 总线匹配成功后调用 probe
- 驱动在 probe 中创建具体设备实例
- 绑定操作函数表
- 框架不关心具体实现类型

六、模式特征总结

在嵌入式 C 中，工厂方法通常具备：

- 初始化函数 / probe 函数
- 注册机制
- 函数指针表实现多态
- 框架与驱动解耦

表现形式不是：

- 继承
- virtual 函数

而是：

结构体 + 函数指针 + 注册机制

七、优点

- ✓ 解耦框架与驱动
- ✓ 易于扩展新设备
- ✓ 支持模块化加载（Linux）
- ✓ 符合分层架构设计

八、代价

- ✗ 引入函数指针复杂度
- ✗ 调试路径更长
- ✗ 代码理解成本增加

在资源极小 MCU 上需要权衡。

九、总结

在嵌入式系统中：

工厂方法并不是“类继承 + virtual”的形式，
而是通过“驱动初始化 / probe + 函数指针表”实现对象创建的延迟与解耦。

RT-Thread 设备模型和 Linux 驱动模型是成熟的工业级实现。

补充：RT-Thread 与 Linux 是否属于工厂方法？

在嵌入式实际工程中，工厂方法通常不会以“继承 + virtual”的形式出现，而是以**驱动注册机制**或 **probe 机制**实现。

1 Linux 驱动模型 —— 典型工厂方法

核心流程：

```
设备匹配成功
    ↓
调用 driver->probe()
    ↓
probe 创建具体设备实例
    ↓
绑定操作函数表
```

特点：

- 框架定义创建时机
- 具体驱动实现创建逻辑（probe）
- 框架不直接实例化具体设备
- 创建与使用分离

结论：

Linux driver model 是工程化实现的典型工厂方法。

2 RT-Thread 设备框架 —— 注册式工厂方法

核心流程：

```
驱动初始化
    ↓
构建设备对象
    ↓
绑定 ops 表
    ↓
rt_device_register()
```

特点：

- 设备对象由具体驱动构造
- 框架只负责统一管理和调用
- 上层通过抽象接口访问

与 Linux 不同之处：

- Linux 是“框架调用子类创建”
- RT-Thread 是“驱动主动注册”

但本质相同：

创建逻辑由具体实现决定，而非框架统一实例化。

结论：

RT-Thread 属于注册式工厂方法变体。